

Data Pipelines

3 important definitions

A data pipeline is an end-to-end process for moving raw data from one or more sources to a destination. The data is processed and transformed along the way so it can be stored, analyzed, and used for business intelligence or machine learning.

Data orchestration is the automated process of managing, scheduling, and coordinating multiple data pipelines and tasks to run efficiently and in the correct order. It is a higher-level concept that focuses on the overall management of a data ecosystem.

A Directed Acyclic Graph (DAG) is a mathematical data structure that uses nodes and directed edges to represent a sequence of tasks. In data engineering, a DAG is used as a blueprint for a workflow, defining the order of execution and dependencies between tasks.

Introducing Data Pipelines

Manage data flow from collection to visualization

Ensure efficiency and minimize blockages

Example: Weather dashboard pipeline



Steps:

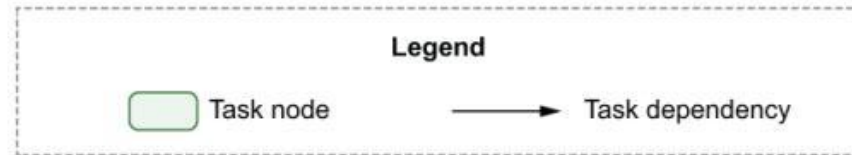
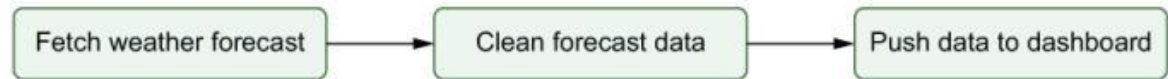
1. Fetch weather forecast data from a weather API.
2. Clean or otherwise transform the fetched data (e.g., converting temperatures from Fahrenheit to Celsius or vice versa), so that the data suits our purpose.
3. Push the transformed data to the weather dashboard

Pipeline as a Graph

Tasks represented as
nodes, dependencies
as edges

Directed Acyclic Graph
(DAG) prevents loops

Defines correct
execution order



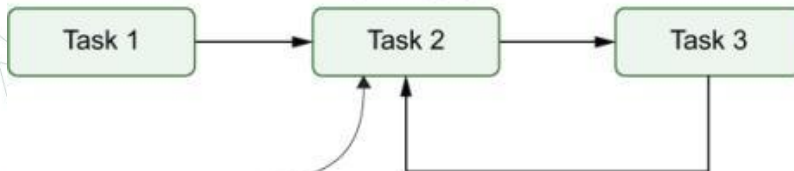
Circular Dependencies

Cyclic graphs cause deadlocks
Tasks depend on each other and
cannot execute
DAG avoids such execution issues

A directed *acyclic* graph (DAG) of tasks



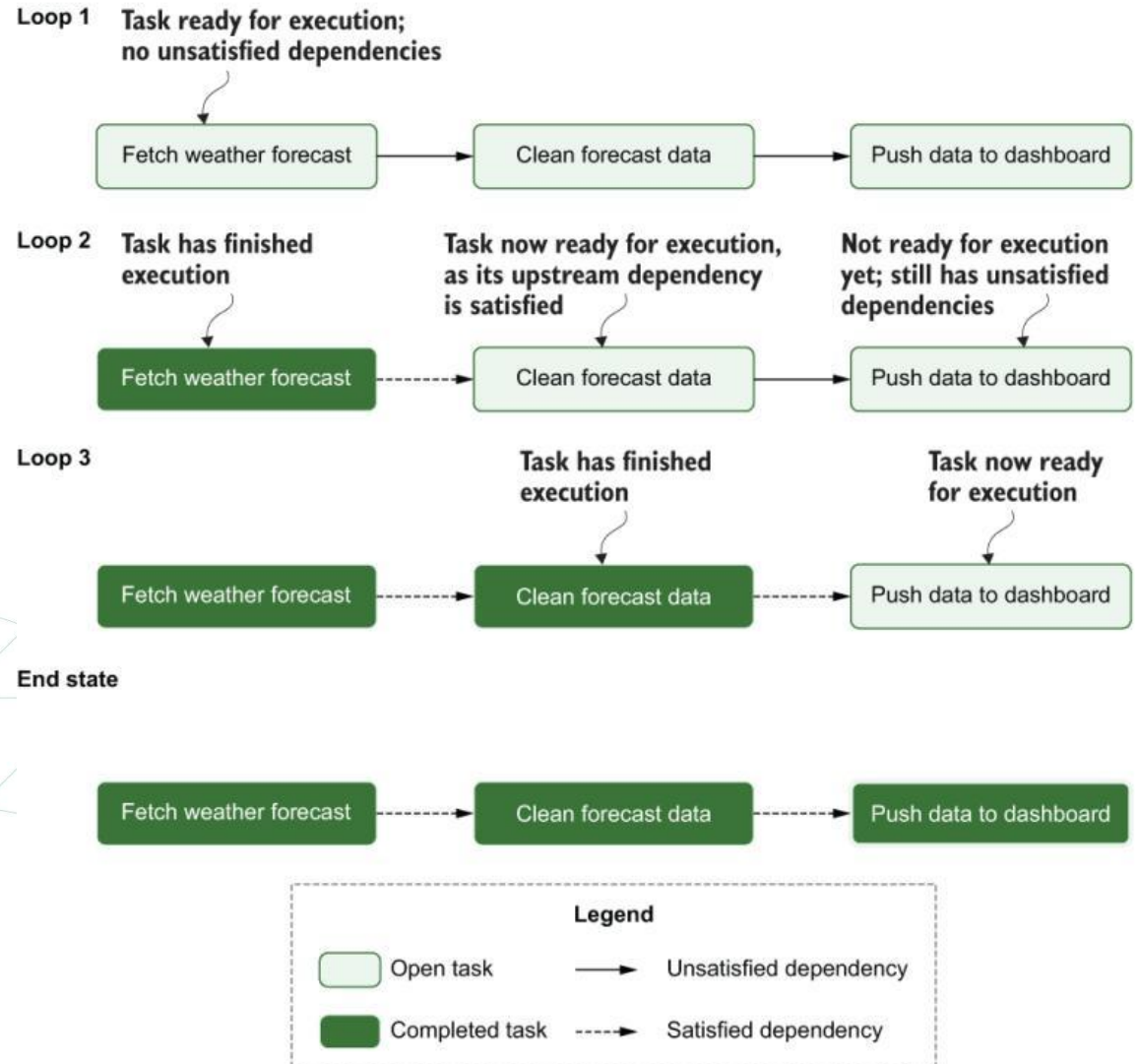
A directed *cyclic* graph of tasks



Task 2 will never be able to execute,
due to its dependency on task 3,
which in turn depends on task 2.

Executing a DAG

Algorithm checks upstream dependencies
Tasks queued only when dependencies are satisfied
Repeat until all tasks complete

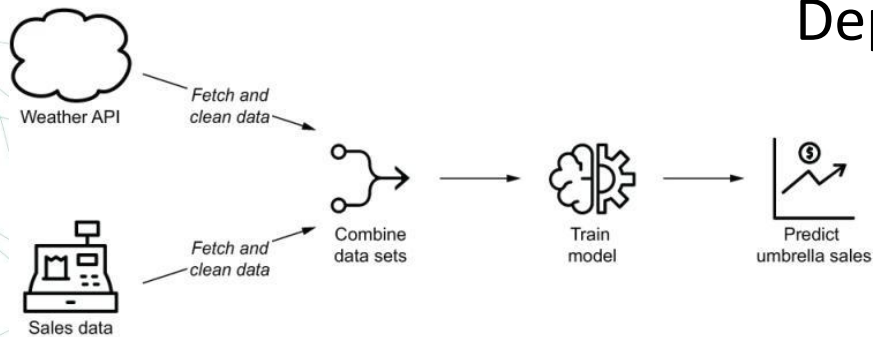


Executing a DAG Algorithm

1. For each open (= uncompleted) task in the graph, do the following:
 - 1.1 For each edge pointing toward the task, check if the “upstream” task on the other end of the edge has been completed.
 - 1.2 If all upstream tasks have been completed, add the task under consideration to a queue of tasks to be executed.
2. Execute the tasks in the execution queue, marking them completed once they finish performing their work.
3. Jump back to step 1 and repeat until all tasks in the graph have been completed.

Umbrella Demand Forecast (Case Study)

Combine weather and sales data
Train ML model to predict umbrella
demand
Deploy model for business use



1. Prepare the sales data by doing the following:
 - Fetching the sales data from the source system
 - Cleaning/transforming the sales data to fit requirements
2. Prepare the weather data by doing the following:
 - Fetching the weather forecast data from an API
 - Cleaning/transforming the weather data to fit requirements
3. Combine the sales and weather data sets to create the combined data set that can be used as input for creating a predictive ML model.
4. Train the ML model using the combined data set.
5. Deploy the ML model so that it can be used by the business.

Independent Tasks in DAGs

Sales and weather tasks are independent

Can run in parallel to save time

Graph shows separate branches



Graph-Based Representation Properties

Clarity: Visualizes tasks and dependencies in an intuitive way

Acyclic nature: Prevents circular dependencies and deadlocks

Parallelism: Independent branches can run simultaneously

Incremental tasks: Easier to isolate and rerun failed tasks

Scalability: Handles complex workflows better than linear scripts

Pipeline graphs vs. sequential scripts

Feature	Sequential Scripts	Pipeline Graphs (DAGs)
Structure	Linear execution; instructions run one after another.	Task network driven by data flow; dependencies define potential parallel execution.
Simplicity	Intuitive and easy to implement for small, straightforward workflows.	Higher entry barrier; requires specialized tooling, though visual DAGs enhance clarity.
Workflow	Rigid, sequential; execution order dictated by code structure.	Flexible; enables parallelization where dependencies permit, adaptable to diverse workflow goals.
Dependencies	Implicit; each step relies on prior completion.	Explicit; tasks declare dependencies, ensuring transparent data and process flow.
Reusability	Low modularity; modifications often require duplication or restructuring.	High modularity; tasks/components are portable, reusable, and easily interchangeable across pipelines.

Pipeline graphs vs. sequential scripts

Error Handling

Failures often cascade; requires custom error handling or manual intervention.

Resilient by design; supports task-level error handling, retries, and recovery strategies.

Scalability

Limited; sequential execution becomes a bottleneck for large datasets or complex operations.

Engineered for scale; supports distributed, parallel execution across large workloads.

Visuals

None inherent; visualization requires manual effort or external documentation.

Visual DAGs are core; provide transparency and aid collaboration across teams.

Use Case

Best for simple data processing, administrative tasks, and repetitive automation.

Ideal for multi-stage data workflows, CI/CD, ML pipelines, and orchestration-heavy processes.

Tools

Lightweight: text editors, scripting languages (Python, Bash).

Enterprise-ready: workflow orchestrators (Airflow, Dagster, Prefect, Kubeflow).

Introducing Apache Airflow



Apache
Airflow

Apache Airflow is an open source tool for programmatically authoring, scheduling, and monitoring data pipelines. Every month, millions of new and returning users download Airflow and it has a large, active open source community. The core principle of Airflow is to define data pipelines as code, allowing for dynamic and scalable workflows.



Apache
Airflow



History of Apache Airflow

Apache Airflow originated at Airbnb in 2014 by Maxime Beauchemin to manage complex data pipelines, becoming open-source in 2015 under the Apache License 2.0. It joined the Apache Software Foundation Incubator in 2016 and became a Top-Level Project in 2019, establishing it as a standard for data workflow orchestration.

Problem:

Airbnb faced growing complexity in managing its data workflows, with existing tools proving too rigid or unable to handle the dynamic nature of modern data pipelines.

Solution:

Maxime Beauchemin, a data engineer at Airbnb, developed Airflow to address these issues, creating a flexible, programmable way to define workflows as code using Python.



Introducing Apache Airflow

Apache Airflow is a platform for programmatically authoring, scheduling, and monitoring workflows. It is especially useful for creating and orchestrating complex data pipelines.

Data orchestration sits at the heart of any modern data stack and provides elaborate automation of data pipelines. With orchestration, actions in your data pipeline become aware of each other and your data team has a central location to monitor, edit, and troubleshoot their workflows.

One of the core tenets of Airflow that sets it apart from other data orchestration tools is defining data pipelines as code. Modern data teams seek to define their workflows in code to:

- Manage workflows with their existing change-control and version-control tooling.
- Bulk-create/update workflows too numerous, large, or dynamic to manage from a GUI.
- Define re-usable parameterizable workflow sections.

Apache Airflow Use Cases

ETL/ELT: 86% of Airflow users use it for Extract-Transform-Load (ETL) and Extract-Load-Transform (ELT) patterns. Often, these pipelines support critical operational processes.

Business operations: 58% of Airflow users have used Airflow to orchestrate data supporting their business directly, creating data-powered applications and products, often in combination with MLOps pipelines.

MLOps and GenAI: 23% of Airflow users are already orchestrating Machine Learning Operations (MLOps) with Apache Airflow and 9% use Airflow specifically for GenAI use cases.

Managing infrastructure: Airflow can be used to spin up and tear down infrastructure. For example, to create and delete temporary tables in a database or spin up and down a Spark cluster. This type of use case has been implemented by 18% of Airflow users.

Code Example

```
1 from airflow.contrib.sensors.file_sensor import FileSensor
2 from airflow.operators.dummy_operator import DummyOperator
3
4 import datetime
5 from datetime import date, timedelta
6 import airflow
7
8 default_args = {
9     "depends_on_past": False,
10    "start_date": airflow.utils.dates.days_ago(1),
11    "retries": 1,
12    "retry_delay": datetime.timedelta(hours=5),
13 }
14 today = datetime.datetime.today()
15 yesterday = date.today() - timedelta(days=1)
16
17 with airflow.DAG("file_sensor_example", default_args=default_args,
18                 schedule_interval= "*/5 * * * *") as dag:
19
20     start_task = DummyOperator(task_id="start")
21     stop_task = DummyOperator(task_id="stop")
22     sensor_task = FileSensor(task_id="file_sensor_task",
23                             poke_interval=30,
24                             fs_conn_id= "<path>",
25                             filepath= "<file or directory name>")
26
27 start_task >> sensor_task >> stop_task
```

Imported Libraries

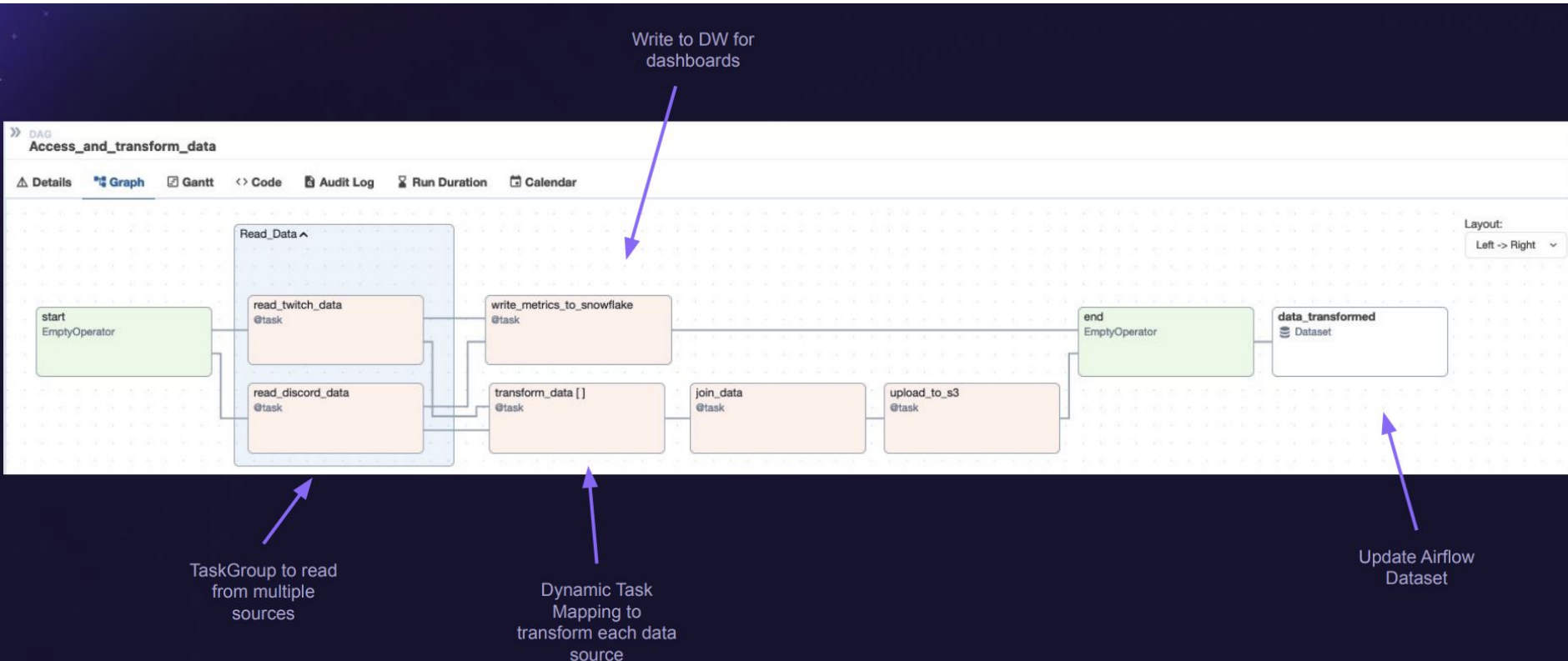
Default Arguments/dictionary of default parameters

Unique identifier + schedule interval

Task definitions

Dependency

Apache Airflow DAG GUI



Apache Airflow DAGs GUI

DAGs

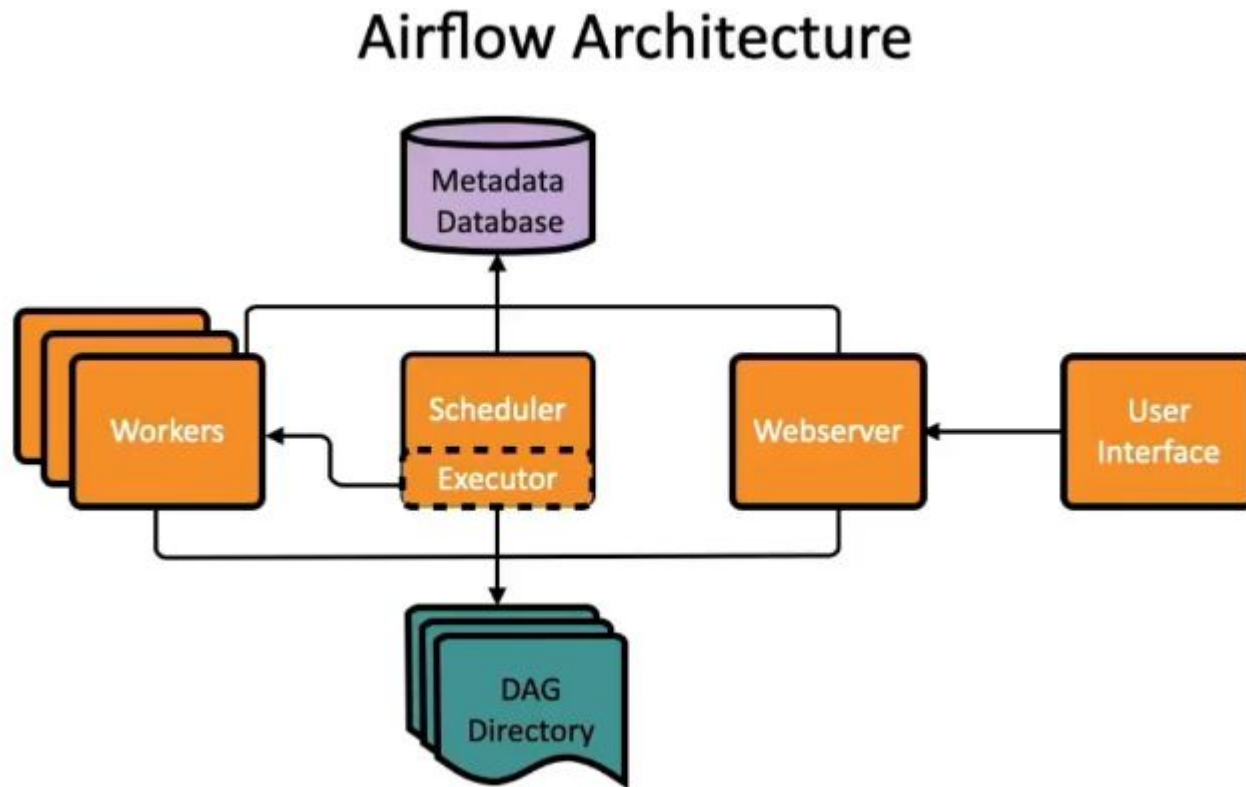
All **26** Active **10** Paused **16**

Filter DAGs by tag

Search DAGs

 DAG	Owner	Runs 	Schedule	Last Run 	Recent Tasks 	Actions	Links
☒ example_bash_operator example example2	airflow		0 0 ***	2020-10-26, 21:08:11 		  	...
☒ example_branch_dop_operator_v3 example	airflow		* / 1 * * * *			  	...
☐ example_branch_operator example example2	airflow		@daily	2020-10-23, 14:09:17 		  	...
☒ example_complex example example2 example3	airflow		None	2020-10-26, 21:08:04 		  	...
☒ example_external_task_marker_child	airflow		None	2020-10-26, 21:07:33 		  	...
☒ example_external_task_marker_parent	airflow		None	2020-10-26, 21:08:34 		  	...
☒ example_kubernetes_executor example example2	airflow		None			  	...
☒ example_kubernetes_executor_config example3	airflow		None	2020-10-26, 21:07:40 		  	...
☒ example_nested_branch_dag example	airflow		@daily	2020-10-26, 21:07:37 		  	...
☐ example_passing_params_via_test_command example	airflow		* / 1 * * * *			  	...

Apache Airflow Architecture Components



Apache Airflow Architecture Components

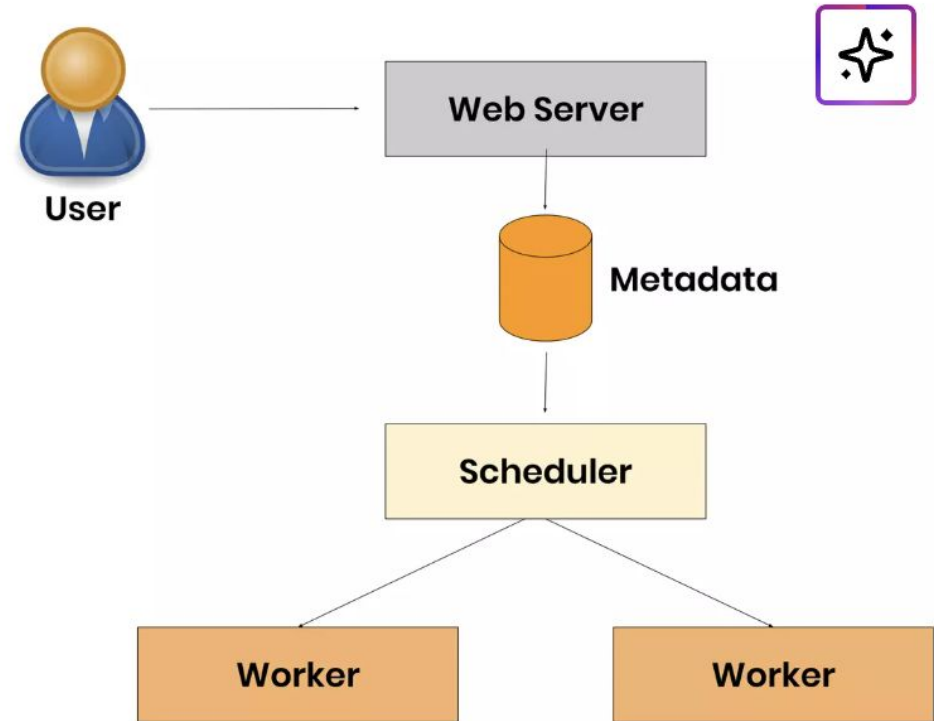
- A [scheduler](#), which handles both triggering scheduled workflows, and submitting [Tasks](#) to the executor to run.
- The [executor](#), is a configuration property of the *scheduler*, not a separate component and runs within the scheduler process. There are several executors available out of the box, and you can also write your own.

Apache Airflow Architecture Components

- A webserver, which presents a handy user interface to inspect, trigger and debug the behaviour of DAGs and tasks.
- A folder of DAG files, which is read by the *scheduler* to figure out what tasks to run and when to run them.
- A metadata database, which airflow components use to store state of workflows and tasks. Setting up a metadata database is described in [Set up a Database Backend](#) and is required for Airflow to work.

The workflow under the hood

1. A user manages/schedules DAGs using Airflow
2. Airflow webserver stores scheduling metadata in metadata DB
3. Scheduler picks up new schedule and distributes work
4. Workers pick up Airflow Task



Workloads

A DAG runs through a series of Tasks, and there are three common types of task you will see:

Operators, predefined tasks that you can string together quickly to build most parts of your DAGs.

Sensors, a special subclass of Operators which are entirely about waiting for an external event to happen.

A **TaskFlow**-decorated `@task`, which is a custom Python function packaged up as a Task.

Control Flow

DAGs are designed to be run many times, and multiple runs of them can happen in parallel. DAGs are parameterized, always including an interval they are “running for” (the data interval), but with other optional parameters as well.

Tasks have dependencies declared on each other. You’ll see this in a DAG either using the `>>` and `<<` operators:

```
first_task >> [second_task, third_task]
fourth_task << third_task
```

Or, with the `set_upstream` and `set_downstream` methods:

```
first_task.set_downstream([second_task, third_task])
fourth_task.set_upstream(third_task)
```

Dataflow

To pass data between tasks you have three options:

XComs (“Cross-communications”), a system where you can have tasks push and pull small bits of metadata.

Uploading and downloading large files from a storage service (either one you run, or part of a public cloud)

TaskFlow API automatically passes data between tasks via implicit XComs

Airflow vs. dbt

Feature	Apache Airflow	dbt
Primary Function	Workflow orchestration	Data transformation within data warehouses
Language	Python	SQL with Jinja templating
Use Case	Scheduling and managing complex workflows	Transforming raw data for analysis
Ideal Users	Data engineers	Data analysts and analytics engineers
Scheduling Capabilities	Built-in	Available in dbt Cloud; dbt Core requires external scheduler
Integration	Can orchestrate dbt runs	Focused on transformations; can be triggered by Airflow



Thank you!